

Last updated: January 16, 2023

COMP 3000 (WINTER 2023) OPERATING SYSTEMS

TUTORIAL 2

Tasks/Questions

Part A

1. Looking at the `.s` file produced from `gcc -S -O2 hello.c`, do you see anything familiar and discussed in the lectures of our first week?
You can see sections like `.text` and `.rodata`.
2. Where (e.g., at which line) is it supposed to call that `printf()`? If it is not doing that, why? (you can just mention what you think/believe)
You may only see `puts()` being called, due to the compiler optimization (you are not passing any arguments which `printf()`'s format string is good at handling so `puts()` will be enough). You can avoid this by either passing an argument to `printf()` or using a compiler option like `-fno-builtin`.
3. Generate the `.s` file for [syscall-hello.c](#) as well. Observing the two `.s` files, what do you think is the key difference?
It calls the `syscall()` library call instead of `printf()`.
4. You can play around by trying to see: how a function in C is reflected in assembly; how to return from a function; how variables/literals are represented; what are **symbols** (refer to commands like `nm`); etc.
We will explain a bit in class.

Part B

1. By default, gcc-produced binaries are dynamically linked (so at runtime they will require dynamic libraries to be present on the system). To compile a binary that is statically linked (so it has no external runtime library dependencies), instead do this:

```
gcc -O2 -static hello.c -o hello.static
```


Then compile a dynamically linked version:

```
gcc -O2 -z lazy hello.c -o hello.dynamic
```


How does the size of `hello.dynamic` compare with that of `hello.static`? Why?
The static one should be much bigger than the dynamic one, since it has all required libraries included. The actual size would vary with many factors (e.g., the compiler/linker version).
2. Compile the other flavor of `hello.c`: [syscall-hello.c](#). Build one static version and one dynamic version as well.

Now considering `hello.dynamic`, `hello.static`, `syscall-hello.dynamic` and `syscall-hello.static`, see what system calls each program produces by running `strace -o sys-somename.log ./somename.static` (or `./somename.dynamic`). Which version generates more system calls? Why?

Note: system calls are saved in the log file `sys-somename.log`. Feel free to save them in a different file for each version.

Dynamically linked programs (containing fewer functions supposedly) do not necessarily generate fewer system calls. It depends on what the code does. For example, I/O-intensive functions need to talk to the OS a lot (to access resources like files/devices).

Specifically, in the context of this question, the code per se does not do much (i.e., only prints once), so offloading it to libraries or including it does not make much difference. However, loading a dynamic library into the address space involves system calls, e.g., `openat()` and `read/pread()` to read metadata from `libc.so` and `mmap()` to map the actual functions for invocation.

3. See what library calls each program produces by running `ltrace -o lib-somename.log ./somename.dynamic` (or `./somename.static`). Which version generates more library calls? Why?

The statically linked version should make no library calls by design. This is the purpose of a static build, which is to ensure that all the libraries are present and that they are the correct version, hence no such dependency issues. The output of `ltrace` can be misleading, which simply says sections describing library dependencies are not found, but expected.

4. Remember when building `hello` in Tutorial 1 you did not use `-z lazy`. Comparing `hello` with `hello.dynamic`, any difference? Why this difference?
Lazy binding leaves the actual resolution of library calls to the runtime so that you can observe them with `ltrace`. With the default “now” binding, it is done at load time, so nothing will be seen at runtime.
5. Use `ldd` to find out what dynamic library dependencies dynamic versions have. What about static versions?

Part C

1. Why are the addresses inconsistent between runs?
Simple answer: because they are just virtual addresses, determined by the OS.
Accurate answer: because of ASLR (address space layout randomization). This will be briefly discussed in class.
You can disable it (not recommended for systems in real use) with:
`echo 0 | sudo tee /proc/sys/kernel/randomize_va_space`
2. Roughly where does the stack seem to be? The heap? Code? Global variables? (hints: recall the memory image layout of a process discussed in the lecture and you can search for a more detailed one somewhere; local variables go to the stack; initialized **data** and global variables go to the data segment, data allocated at runtime go to the heap.)
Local variables are: `lmsg` and `buf`, so `&lmsg` and `&buf` (the location of them) are around the stack. The value of `buf[0]` (dynamically allocated) is in the heap. We will discuss it in a bit more detail.

3. Change each `malloc()` call to allocate more than 128K. What happens to the values of `sbrk`? Why? (Hint: use `strace`)

When the amount to allocate exceeds a certain threshold `malloc()` stops using `sbrk()` and uses `mmap()` to allocate memory instead.